Credit Card Fraud Detection

Adaptive Computation and Machine Learning-COMS7047A

Author 1 - Student Number

Author 2 - Student Number 2

Author 3 - Student Number 3

Author 4 - Student Number 4

May 19, 2026

University of the Witwatersrand

Preface

This report presents a machine learning analysis of the public Credit Card Fraud Detection dataset. The report was written to reflect the project workflow implemented in the source code. It includes the dataset source, preprocessing decisions, train/validation/test methodology, classical supervised models, a supervised multilayer perceptron (MLP), an autoencoder anomaly detector, tuning experiments, training and evaluation figures, final test results, and a distinction between external sources and work created for this project.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Credit card fraud detection is an important machine learning problem because fraudulent transactions can create direct financial losses for customers, merchants, and financial institutions. The task is also technically challenging because fraud cases are rare compared with legitimate transactions. A model that predicts every transaction as legitimate can achieve high accuracy, but it is useless because it detects no fraud.

The aim of this project is to investigate machine learning methods for detecting fraudulent credit card transactions in a highly imbalanced public dataset. The project compares classical supervised classifiers, a supervised Dense Neural Network classifier, and an autoencoder-based anomaly detection method. The main objective is not simply to maximise accuracy, but to find a useful balance between detecting fraud and limiting false alarms.

The objectives of the project are:

- source and document a public dataset of sufficient complexity;
- preprocess the data without leaking validation or test information into training;
- create a stratified train/validation/test split;
- train and compare classical supervised and neural-network models;
- tune decision thresholds using validation data;
- evaluate the final selected models once on the held-out test set;
- present results using both metrics and figures.

The modelling workflow, preprocessing code, threshold tuning, model comparison, final evaluation, and figures were created for this project using Python libraries. The dataset itself is an external public dataset obtained from Kaggle and originally associated with the Machine Learning Group at Universite Libre de Bruxelles.

Chapter 2

Dataset Description and Data Understanding

2.1 Dataset Source

The dataset used in this project is the Credit Card Fraud Detection dataset, available on Kaggle at:

`https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud`

The dataset contains anonymised credit card transactions made by European cardholders in September 2013. It is public and anonymised, so no ethics clearance is required for this project.

2.2 Dataset Contents

The raw dataset contains 284,807 transactions and 31 columns. The target variable is `Class`, where class 0 represents a legitimate transaction and class 1 represents a fraudulent transaction. The input variables are:

- `Time`: seconds elapsed between each transaction and the first transaction in the dataset;
- `Amount`: transaction amount;
- V_1 to V_{28}: anonymised principal components produced by PCA transformation.

The dataset is therefore extremely imbalanced. This has two consequences for modelling. First, accuracy is not a useful main metric. Second, precision-recall behaviour and confusion matrix counts are essential because even a small false-positive rate can produce many false fraud alerts.

Table 2.1: Raw dataset class distribution.

Class	Count	Percentage
Legitimate transactions	284,315	99.827%
Fraudulent transactions	492	0.173%
Total	284,807	100.000%

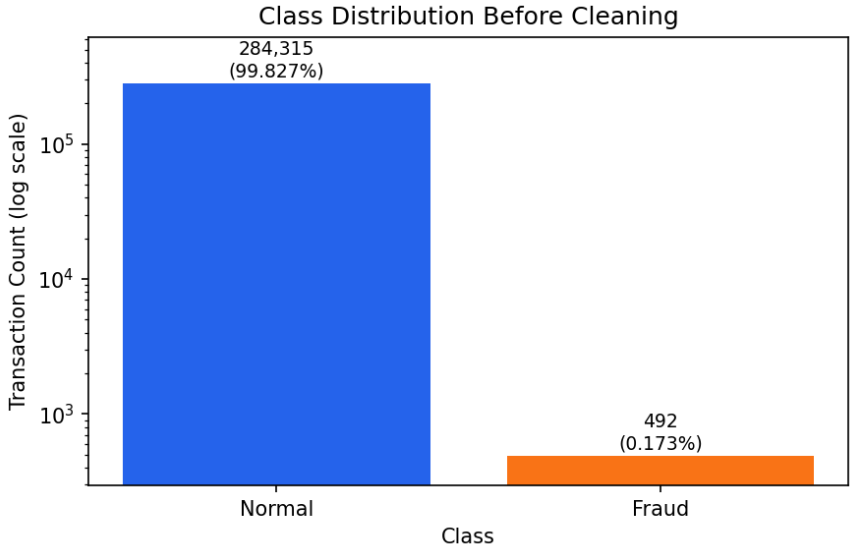

Figure 2.1: Class distribution before cleaning. The log-scaled count axis makes the minority fraud class visible. The imbalance motivates the use of precision, recall, F1-score, ROC-AUC, and especially PR-AUC.

2.3 Focused Data Understanding

Only a focused amount of exploratory analysis was performed because the project goal is modelling rather than extensive EDA. The key aim was to understand properties that affect preprocessing and evaluation.

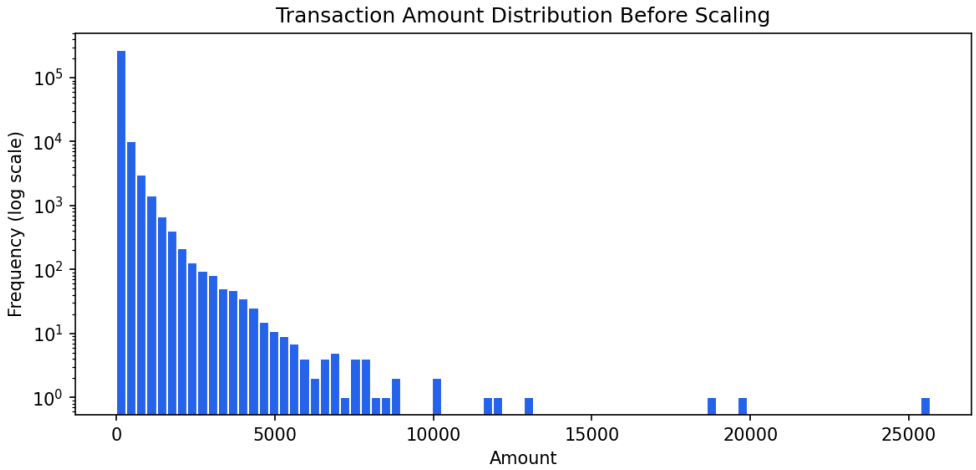

Figure 2.2: Transaction amount distribution before scaling. Amount values are highly skewed, supporting the decision to scale the `Amount` column before training models that are sensitive to feature scale.

Figure 2.3: Transaction amount by class using log-transformed amount values. Fraud and legitimate transactions show some distributional differences, but the overlap is large, so amount alone cannot solve the classification problem.

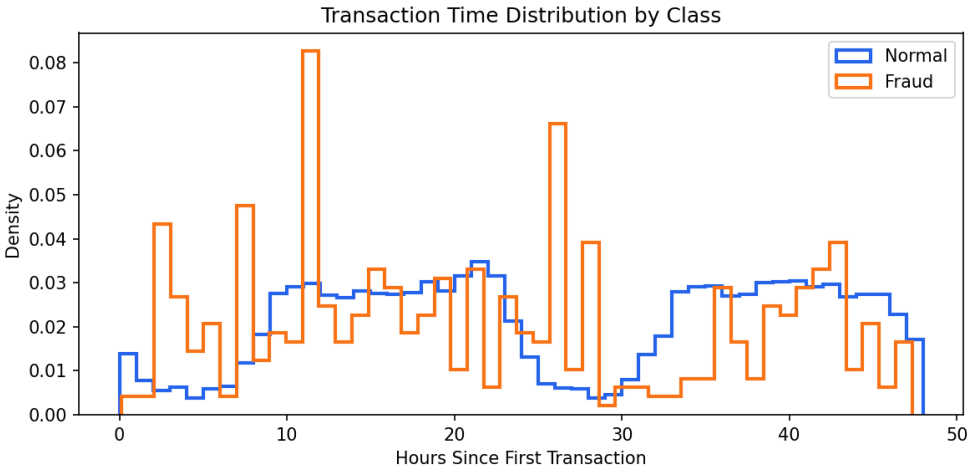

Figure 2.4: Transaction time distribution by class. Time contains some structure, but the class distributions still overlap substantially. This supports treating `Time` as one useful feature rather than a complete separator.

Chapter 3

Data Preprocessing and Train/Validation/Test Split

3.1 Cleaning

The raw dataset was loaded and checked for missing values. No missing values were present, so imputation was not required. Duplicate rows were then removed before splitting the dataset. This step reduced the risk that repeated transactions or repeated transaction patterns could appear in both training and evaluation data.

Table 3.1: Duplicate removal summary.

Quantity	Count
Raw rows	284,807
Duplicate rows removed	1,081
Rows after duplicate removal	283,726
Fraud rows after duplicate removal	473
Legitimate rows after duplicate removal	283,253

3.2 Feature Scaling

Only `Time` and `Amount` were scaled. The features V_1 to V_{28} were already PCA-transformed and were left unchanged.

A `StandardScaler` was fitted on the training split only. The fitted scaler was then applied to the validation and test splits. This prevents validation or test information from influencing preprocessing.

3.3 Train/Validation/Test Split

The cleaned dataset was split using a stratified 70/15/15 split:

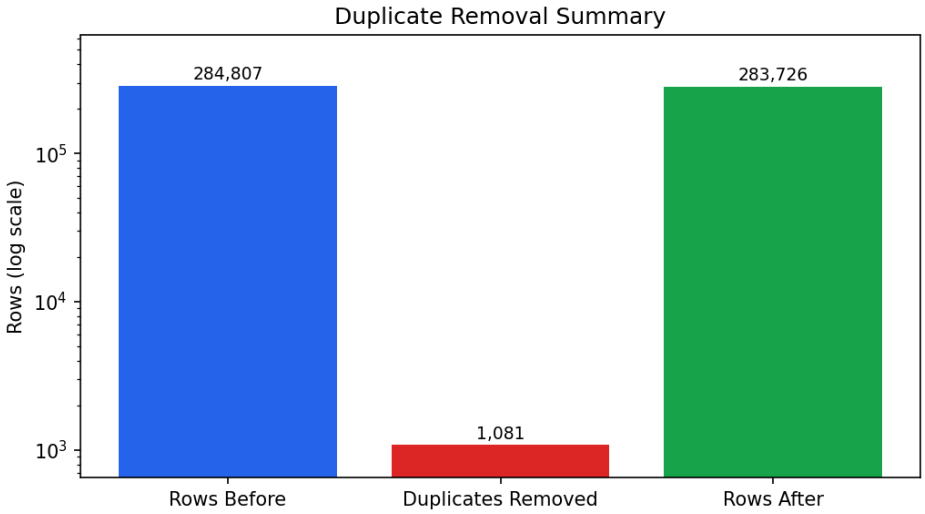

Figure 3.1: Duplicate removal summary. The plot documents the main cleaning step applied before the train/validation/test split.

- training set: used to fit the models;
- validation set: used for model selection, hyperparameter comparison, and threshold tuning;
- test set: used only once for final performance reporting.

Table 3.2: Stratified train/validation/test split.

Split	Total transactions	Fraud transactions	Fraud rate
Training	198,608	331	0.167%
Validation	42,559	71	0.167%
Test	42,559	71	0.167%

3.4 Correlation Structure

A correlation matrix was generated after duplicate removal. This is a normalised covariance-style view, which is more readable than a raw covariance matrix because the dataset mixes PCA features with larger-scale raw features such as `Time` and `Amount`.

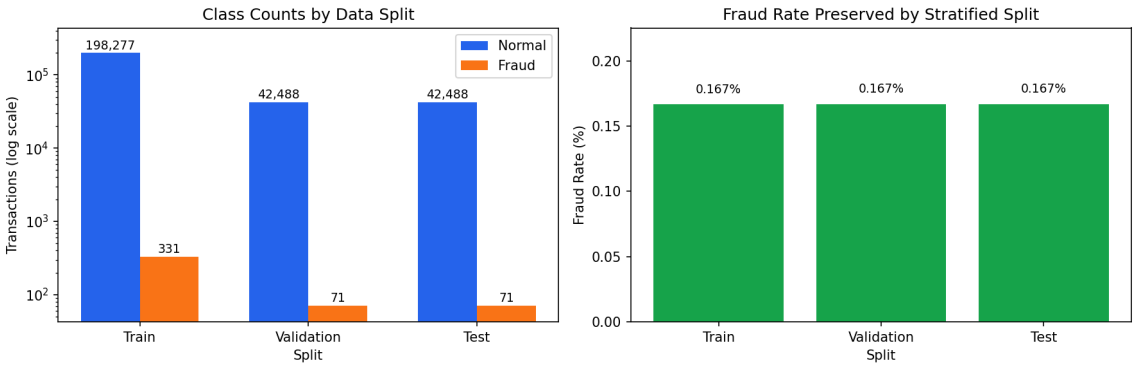

Figure 3.2: Class counts and fraud rate across the train, validation, and test splits. The fraud rate is preserved across splits, confirming that stratification worked as intended.

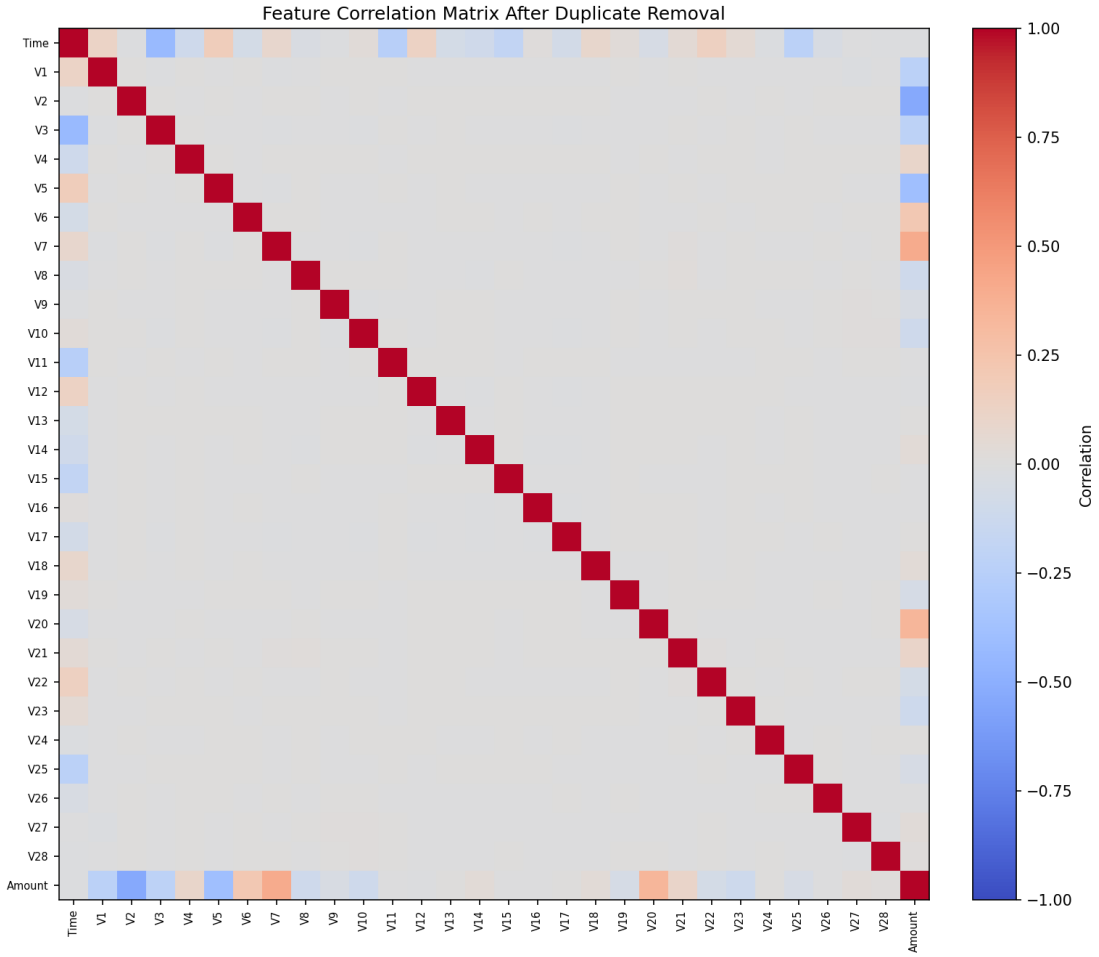

Figure 3.3: Feature correlation matrix after duplicate removal. Most PCA-transformed features have weak pairwise correlation, suggesting limited redundancy among the transformed components.

Chapter 4

Models and Methodology

4.1 Evaluation Metrics

Because fraud is rare, accuracy is not used as the primary metric. A trivial classifier that predicts all transactions as legitimate would be more than 99% accurate but would detect zero fraud cases.

The following metrics were used:

- Precision: among transactions flagged as fraud, the fraction that were actually fraud.
- Recall: among all fraud transactions, the fraction detected by the model.
- F1-score: harmonic mean of precision and recall.
- ROC-AUC: threshold-independent ranking performance over false-positive and true-positive rates.
- PR-AUC: area under the precision-recall curve, especially important for imbalanced data.
- Confusion matrix counts: true negatives, false positives, false negatives, and true positives.

Threshold-dependent metrics were computed using validation-selected thresholds. Final results were then reported on the held-out test set.

4.2 Classical Supervised Models

The supervised models were trained on the labelled training set and evaluated on the validation set. Candidate models included:

- class-weighted Logistic Regression;

- Random Forest with balanced class weights;
- shallow Random Forest with balanced subsampling;
- default Gradient Boosting;
- tuned Gradient Boosting.

The final supervised model was selected from validation results using F1-score, with PR-AUC and recall used as secondary sorting criteria. This selection rule was chosen because the task requires a balance between detecting fraud and limiting false positives.

4.3 MLP Classifier

The MLP classifier is a supervised deep-learning model for the same 30 tabular transaction features. It uses fully connected ReLU hidden layers, dropout regularisation, and a sigmoid output score for fraud probability. Dense layers are appropriate here because each transaction is represented as one fixed feature vector rather than as a customer sequence or text sequence.

Fraud rows are extremely rare in the training split, so the MLP loss uses balanced class weights. For class c, the weight is

$$w_c = \frac{N}{Kn_c}, \tag{4.1}$$

where N is the number of training examples, $K = 2$ is the number of classes, and n_c is the training count for class c. With predicted fraud score p_i and label $y_i \in \{0, 1\}$, the weighted binary cross-entropy objective is

$$\mathcal{L}_{\text{MLP}} = -\frac{1}{N} \sum_{i=1}^{N} \left[w_1 y_i \log(p_i) + w_0 (1 - y_i) \log(1 - p_i) \right]. \tag{4.2}$$

The class weights make fraud mistakes matter during gradient updates, but they also change the score distribution. Therefore the MLP decision threshold is selected on validation data instead of assuming that a score of 0.5 is the best operating point.

4.4 Autoencoder Model

The autoencoder was used as an anomaly detection model. It was trained only on legitimate transactions from the training set. The motivation is that an autoencoder trained on normal behaviour should reconstruct legitimate transactions better than anomalous fraud transactions.

The dense autoencoder architecture was:

```
Input(30) -> Dense(24) -> Dropout -> Dense(12) -> Bottleneck -> Dense(12)
                    -> Dropout -> Dense(24) -> Output(30)
```

The model used ReLU activations in hidden layers, a linear output layer, mean squared reconstruction error, and the Adam optimiser. Let $x_i \in \mathbb{R}^d$ be one transaction vector with $d = 30$ features and let $\hat{x}_i = f_\theta(x_i)$ be its reconstruction. The row-level reconstruction error is

$$e_i = \frac{1}{d} \sum_{j=1}^{d} (x_{ij} - \hat{x}_{ij})^2. \tag{4.3}$$

The autoencoder is fitted only on normal training transactions, denoted $\mathcal{T}_0$, so its reconstruction objective is

$$\mathcal{L}_{\mathrm{AE}} = \frac{1}{|\mathcal{T}_0|} \sum_{i \in \mathcal{T}_0} e_i. \tag{4.4}$$

Candidate bottleneck sizes and dropout rates were evaluated using the validation set. Fraud predictions were made by comparing reconstruction error to a threshold τ: a transaction is flagged when $e_i > \tau$. Larger reconstruction error indicates a more anomalous transaction.

Chapter 5

Experiments and Hyperparameter Tuning

5.1 Supervised Model and Threshold Experiments

For supervised models, probability scores were evaluated over thresholds from 0.05 to 0.95 in increments of 0.05. Threshold tuning was performed only on the validation set. The test set was not used until the final evaluation.

Table 5.1: Best validation result for each supervised model after threshold tuning.

Model	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression, balanced	0.95	0.287	0.873	0.432	0.977	0.720
Random Forest, balanced	0.35	0.937	0.831	0.881	0.970	0.855
Random Forest, shallow	0.60	0.908	0.831	0.868	0.980	0.839
Gradient Boosting, default	0.35	0.889	0.451	0.598	0.584	0.418
Gradient Boosting, tuned	0.15	0.818	0.634	0.714	0.824	0.590

The balanced Random Forest was selected because it achieved the best validation F1-score while maintaining high precision and useful recall. Logistic Regression ranked transactions reasonably well, but its operational precision was too low. At the default 0.5 threshold, it produced 999 false positives on the validation set and had precision of only 0.061.

5.2 MLP Configuration and Threshold Experiments

The MLP experiments varied depth, width, dropout, and Adam learning rate while keeping the corrected split and threshold-tuning rule unchanged. Early stopping monitored validation behaviour during training, and probability thresholds from 0.05 to 0.95 were evaluated on the validation split after each candidate network had been trained.

The smaller 64/32 MLP was selected because it achieved the best validation F1-score. The deeper alternatives reached similar or slightly higher ranking metrics, but their validation precision-recall balance at the selected threshold was weaker.

Table 5.2: Best validation result for each MLP configuration after threshold tuning.

MLP configuration	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC
Dense 64/32, dropout 0.2, lr 10^{-3}	0.95	0.787	0.831	0.808	0.971	0.708
Dense 128/64/32, dropout 0.3, lr 10^{-3}	0.95	0.678	0.831	0.747	0.978	0.711
Dense 128/64/32, dropout 0.3, lr 5×10^{-4}	0.95	0.723	0.845	0.779	0.977	0.700

5.3 Autoencoder Experiments

The autoencoder experiments varied bottleneck size and dropout rate. Candidate thresholds were based on percentiles of the reconstruction errors from normal training transactions. These thresholds were then evaluated on the validation set.

Table 5.3: Best validation result for each autoencoder configuration.

Autoencoder configuration	Percentile	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC
Bottleneck 4, dropout 0.1	99	1.990	0.107	0.817	0.190	0.958	0.457
Bottleneck 6, dropout 0.2	99	3.614	0.113	0.817	0.198	0.963	0.276
Bottleneck 8, dropout 0.2	99	3.382	0.109	0.817	0.193	0.962	0.302
Bottleneck 12, dropout 0.3	99	4.548	0.099	0.704	0.174	0.957	0.253

The selected autoencoder was the bottleneck 6, dropout 0.2 model with the 99th-percentile threshold. This configuration was selected by validation F1-score. Although the bottleneck 4 model had a higher validation PR-AUC, the selected bottleneck 6 model provided the best thresholded precision-recall balance under the project selection rule.

5.4 Autoencoder Learning-Rate Sweep

The structural autoencoder comparison used Adam learning rate 10^{-3}. To test optimiser sensitivity, a separate fixed-architecture sweep reran the bottleneck 6, dropout 0.2 autoencoder with Adam learning rates 10^{-4}, 5×10^{-4}, and 10^{-3}. The same normal-only training data, validation split, percentile thresholds, and F1-based validation selection rule were retained. This sweep is a validation experiment; it does not reuse the test set for model selection.

Table 5.4: Best validation result for the fixed-architecture autoencoder learning-rate sweep.

Learning rate	Percentile	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC
10^{-4}	99	2.112	0.109	0.845	0.193	0.968	0.472
5×10^{-4}	99	4.593	0.102	0.718	0.179	0.957	0.227
10^{-3}	99	1.836	0.117	0.831	0.205	0.968	0.476

The 10^{-3} run produced the strongest validation F1-score and PR-AUC in this controlled sweep, supporting its use in the main autoencoder experiments. The weaker 5×10^{-4} result also shows that optimiser settings can change anomaly-score separation even when the architecture is held fixed.

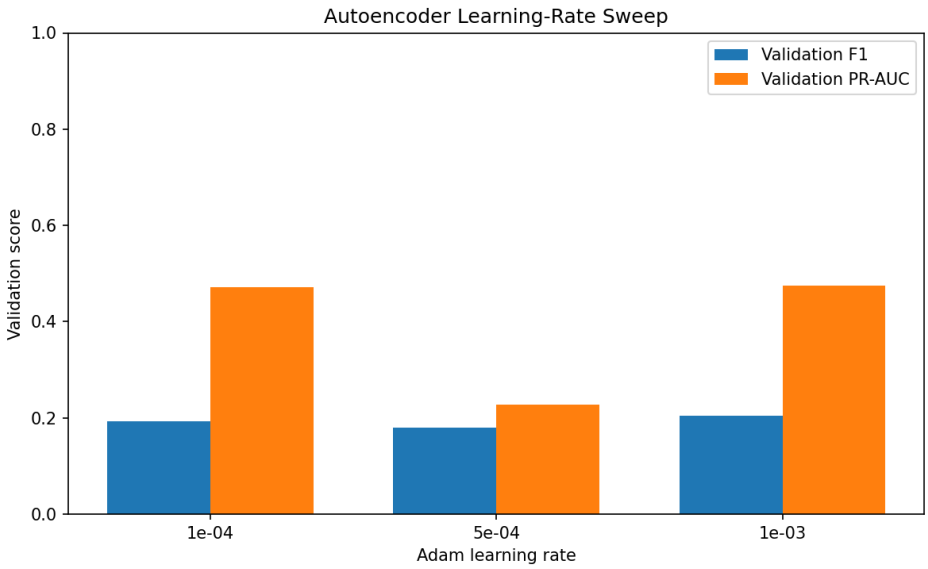

Figure 5.1: Validation comparison for the fixed-architecture autoencoder learning-rate sweep. Each bar uses the best validation threshold for that learning-rate run.

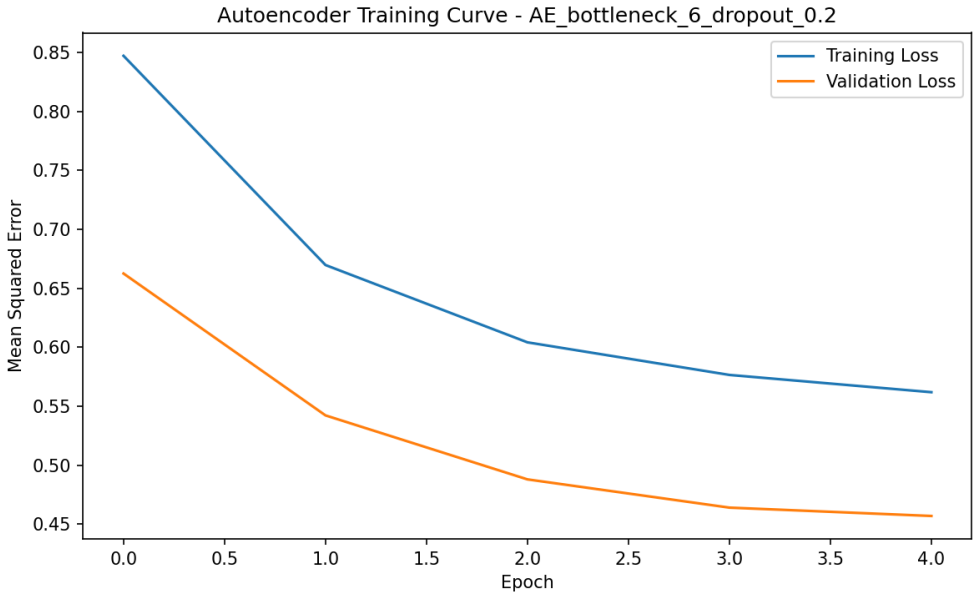

Figure 5.2: Training and validation loss curves for the selected autoencoder. The curves show stable training behaviour and no severe divergence between training and validation loss.

Chapter 6

Results and Analysis

6.1 Random Forest Results

The selected supervised model was the balanced Random Forest with a validation-selected threshold of 0.35. It was evaluated once on the held-out test set.

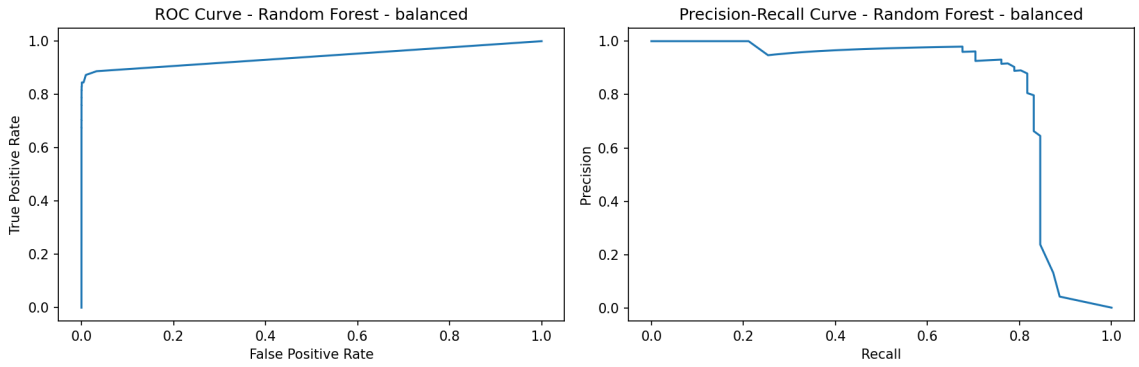

Figure 6.1: ROC and precision-recall curves for the selected Random Forest on the test set. The precision-recall curve is especially important because fraud is rare.

6.2 MLP Results

The selected MLP was the Dense 64/32 network with dropout 0.2, Adam learning rate 10^{-3}, and validation-selected threshold 0.95. It was evaluated once on the held-out test set after the MLP configuration and threshold had been selected on validation data.

6.3 Autoencoder Results

The selected autoencoder used a bottleneck size of 6, dropout of 0.2, and a 99th-percentile reconstruction-error threshold of 3.613886.

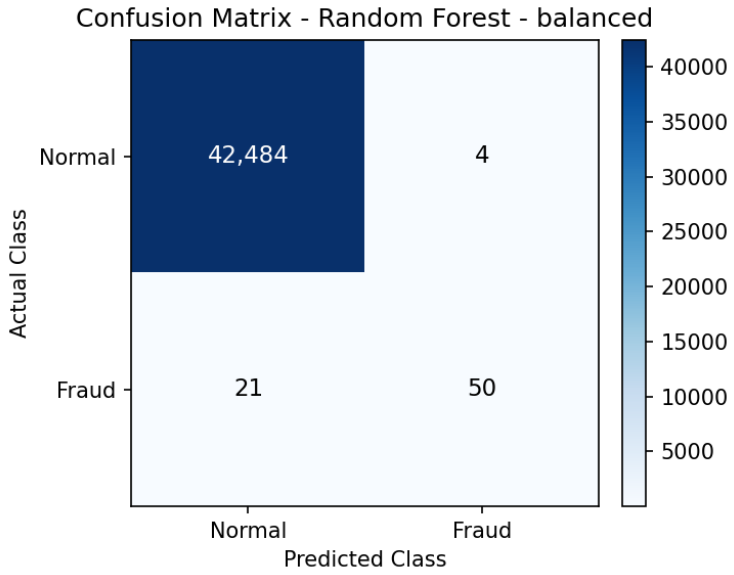

Figure 6.2: Confusion matrix for the selected Random Forest on the test set. The model detects 50 of 71 fraud cases and produces only 4 false positives.

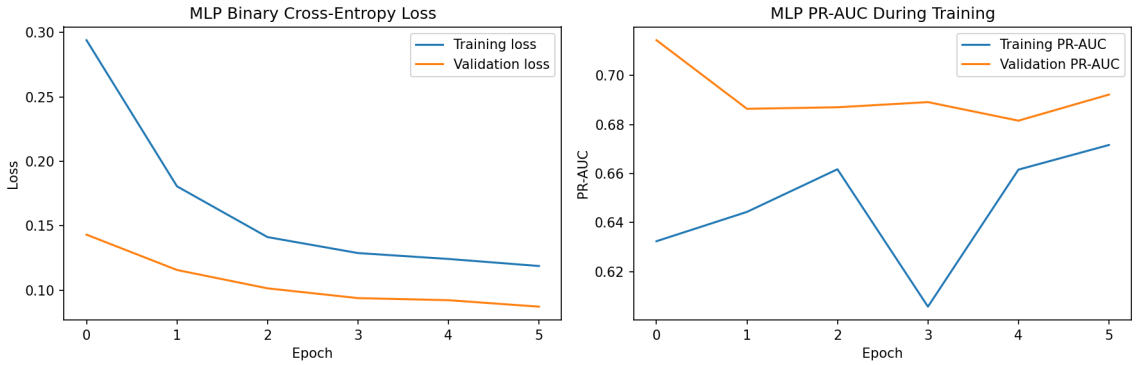

Figure 6.3: Training curves for the selected MLP. The loss curves show optimisation progress, while validation PR-AUC tracks minority-class ranking quality during training.

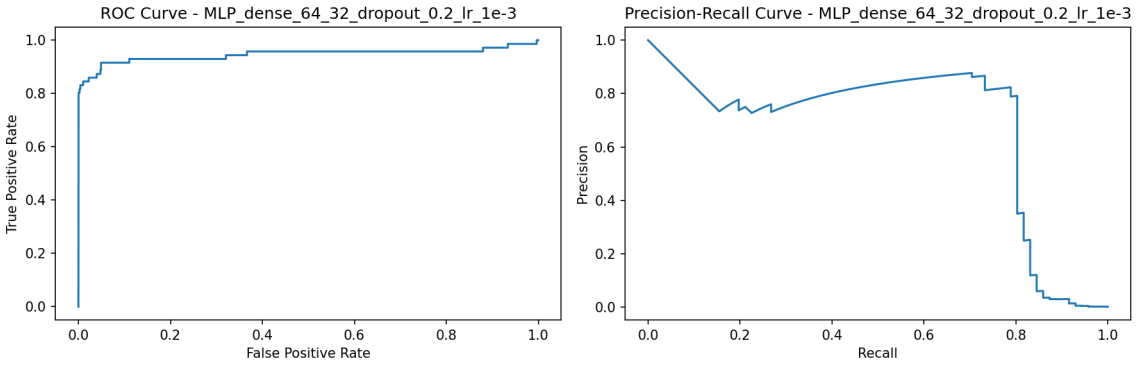

Figure 6.4: ROC and precision-recall curves for the selected MLP on the test set. The PR curve is the more demanding view because only 71 fraud rows are present in the test split.

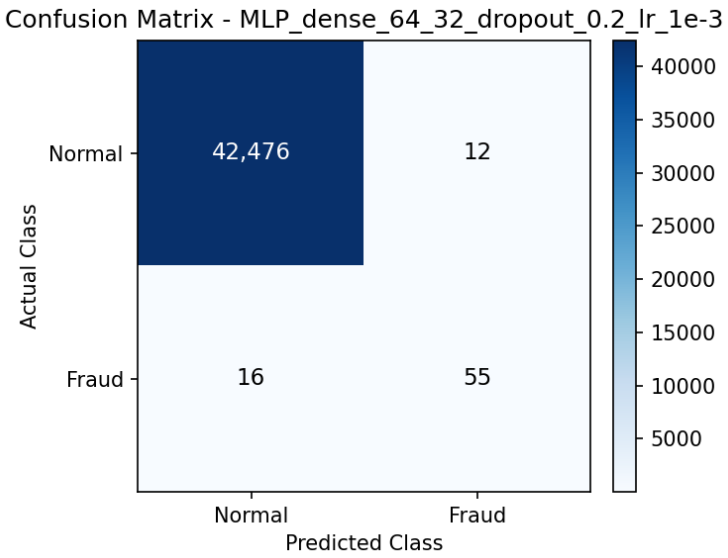

Figure 6.5: Confusion matrix for the selected MLP on the test set. The MLP detects 55 of 71 fraud cases and produces 12 false positives.

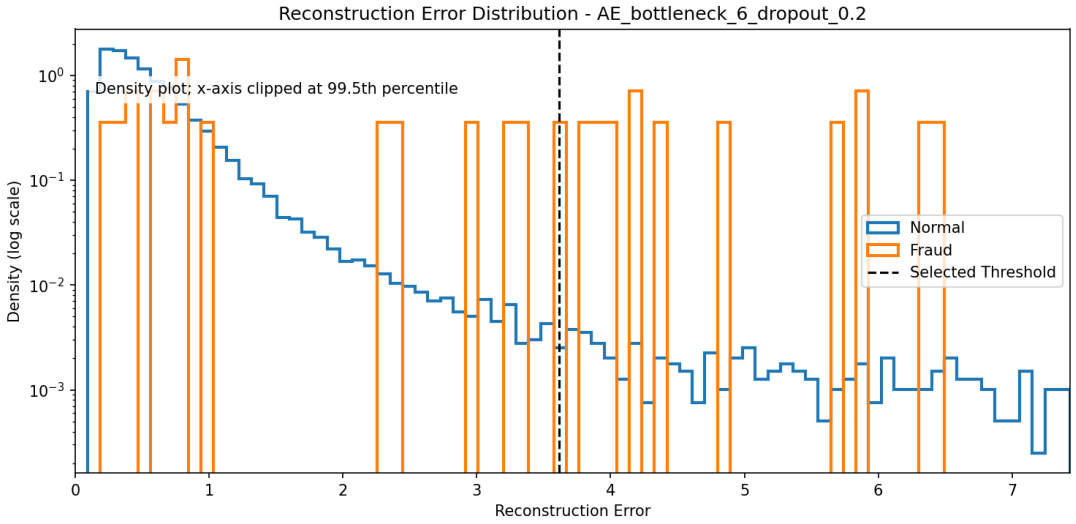

Figure 6.6: Reconstruction error distribution for the selected autoencoder on the test set. The plot uses density and a log-scaled y-axis because the dataset is highly imbalanced. The overlap between normal and fraud errors explains the autoencoder's low precision.

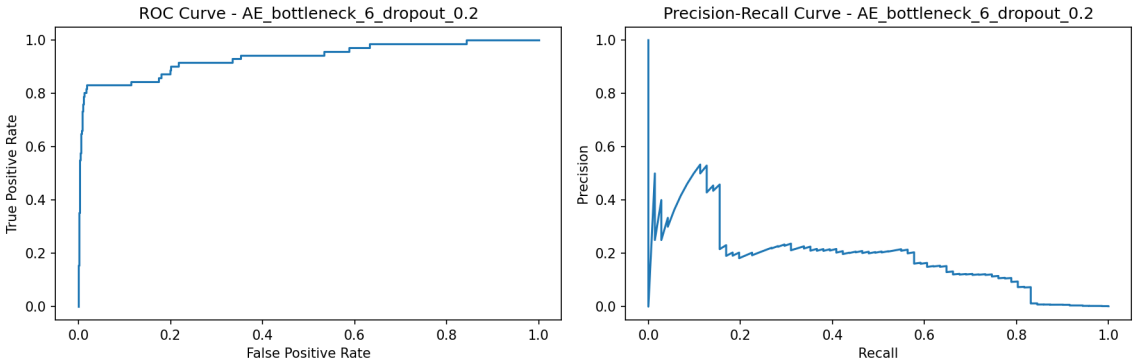

Figure 6.7: ROC and precision-recall curves for the selected autoencoder on the test set. ROC-AUC is high, but PR-AUC is much lower than the Random Forest PR-AUC, showing weaker practical minority-class performance.

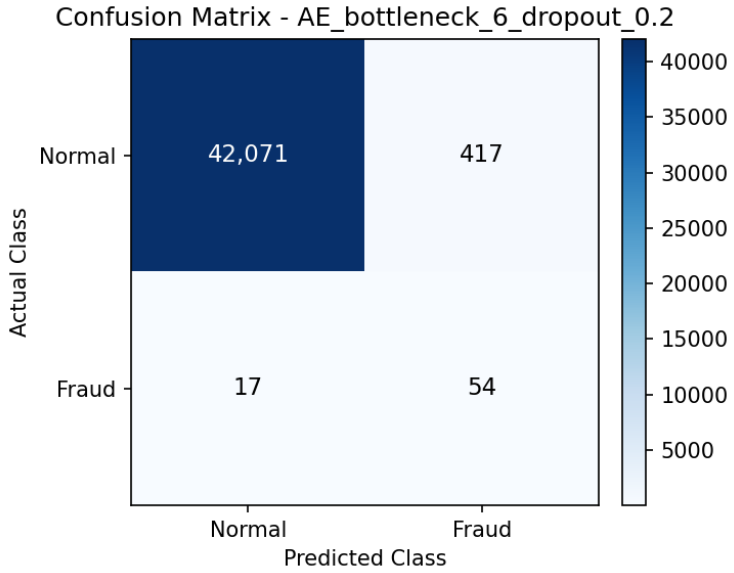

Figure 6.8: Confusion matrix for the selected autoencoder on the test set. The autoencoder detects slightly more fraud cases than Random Forest, but produces many more false positives.

6.4 Final Model Comparison

Table 6.1: Final held-out test results.

Model	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC	TN	FP	FN	TP
Random Forest, balanced	0.350000	0.925926	0.704225	0.800000	0.941271	0.815670	42,484	4	21	50
MLP, Dense 64/32 dropout 0.2	0.950000	0.820896	0.774648	0.797101	0.945893	0.653590	42,476	12	16	55
Autoencoder, bottleneck 6/dropout 0.2	3.613886	0.114650	0.760563	0.199262	0.935140	0.192112	42,071	417	17	54

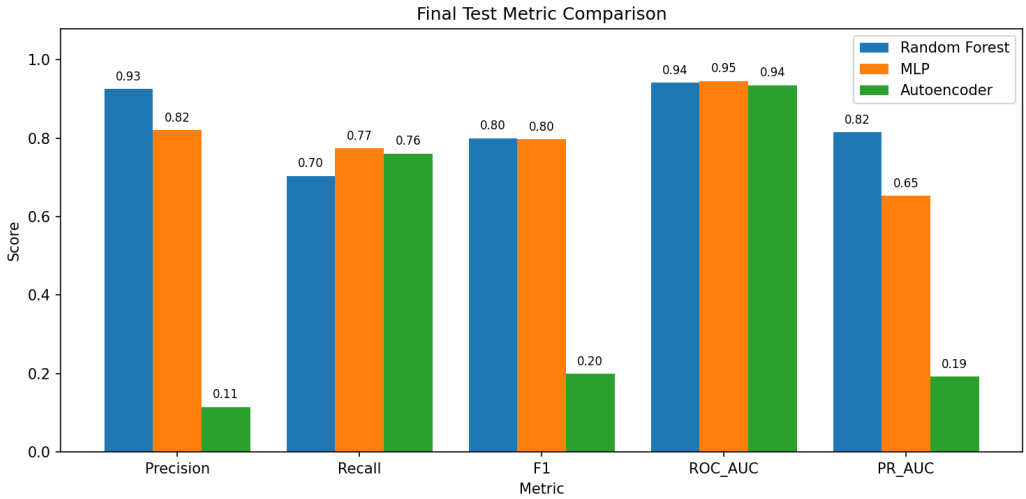

Figure 6.9: Final test metric comparison for Random Forest, MLP, and autoencoder. Random Forest has the strongest F1-score and PR-AUC, while the MLP is a much stronger precision-recall compromise than the autoencoder.

The Random Forest remains the strongest practical model by a narrow F1 margin and a clear PR-AUC margin. It detects 50 of 71 fraud cases while producing only 4 false positives. The class-weighted MLP detects 55 fraud cases and keeps false positives low at 12, so it is a credible supervised deep-learning alternative with higher recall and slightly lower precision than Random Forest. The autoencoder detects 54 fraud cases, but produces 417 false positives, giving it much lower precision and a much weaker F1-score.

This result also shows why PR-AUC is important. The MLP and autoencoder both have ROC-AUC values above 0.93, which suggests useful ranking ability, but their PR-AUC values differ substantially. For rare fraud detection, PR-AUC and confusion matrix counts better reflect the practical cost of false alarms than ROC-AUC alone.

6.5 Mechanistic Interpretation of Model Behaviour

The Random Forest benefits directly from labelled fraud examples. Its trees partition nonlinear interactions among the anonymised PCA features, scaled `Time`, and scaled `Amount`, and the ensemble averages many such partitions. At the validation-selected threshold, only high-confidence regions of feature space are flagged, which explains its very small false-positive count and high precision. Its recall is lower than the MLP recall because this conservative operating point leaves some rare or borderline fraud patterns unflagged.

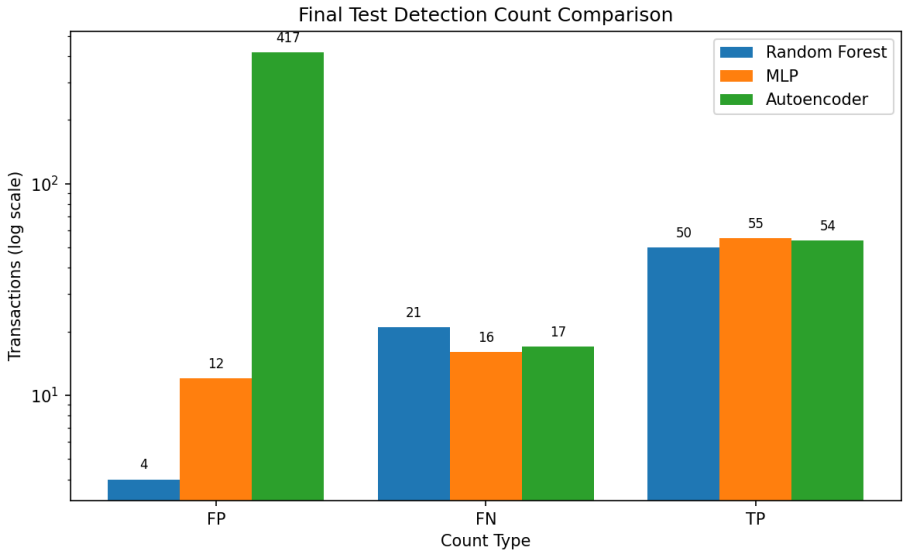

Figure 6.10: Final test detection-count comparison. The MLP detects five more fraud cases than Random Forest with eight additional false positives, whereas the autoencoder detects four more fraud cases than Random Forest with 413 additional false positives.

The MLP also uses labels, but it learns smooth nonlinear feature combinations through gradient updates. Balanced class weights enlarge the loss contribution of the minority fraud rows, so the network is pushed to pay attention to rare fraud patterns that an unweighted classifier could ignore. This helps it detect more fraud cases than Random Forest, but the weighting also shifts positive scores upward for some normal transactions. The high validation-selected threshold of 0.95 is therefore important: it controls false positives after class-weighted training. The remaining PR-AUC gap to Random Forest suggests that the MLP score ranking is less sharply separated for the rare class on the test set.

The autoencoder receives no fraud labels during training. It learns a reconstruction manifold for normal transactions and treats large reconstruction error as anomaly evidence. This mechanism can recover fraud rows that look unusual relative to normal training data, giving useful recall. However, not every fraud transaction is far from the normal manifold in the PCA-transformed feature space, and some legitimate transactions are unusual enough to reconstruct poorly. That overlap in reconstruction errors explains why the autoencoder can have high ROC-AUC but still produce many false positives and low precision at an operational threshold.

6.6 What Did Not Work

Logistic Regression was not selected because, although it achieved strong ranking metrics, its thresholded performance produced too many false positives. At threshold 0.5 it achieved recall of 0.915 on the validation set but precision of only 0.061, producing 999 false positives.

Gradient Boosting did not outperform Random Forest in the experiments. The tuned Gradient Boosting model achieved validation F1-score of 0.714, lower than both Random

Forest configurations.

The MLP met the need for a supervised deep-learning comparison and achieved a final F1-score close to Random Forest. It was not selected as the strongest practical final model because Random Forest retained higher precision and much higher PR-AUC on the test set.

The autoencoder was useful as a complex anomaly detection experiment, but it was not the best final model. Its reconstruction scores detected slightly more fraud cases than Random Forest, but the high false-positive count made it less practical when labelled data was available.

Chapter 7

Conclusion

This project analysed the public Kaggle/ULB Credit Card Fraud Detection dataset using classical supervised learning, a supervised MLP classifier, and autoencoder-based anomaly detection. We removed duplicate rows, used a stratified train/validation/test split, scaled `Time` and `Amount` using a training-only scaler, tuned thresholds on the validation set, and evaluated final models once on the held-out test set.

The best final model was the balanced Random Forest with a threshold of 0.35. On the test set, it achieved precision of 0.926, recall of 0.704, F1-score of 0.800, ROC-AUC of 0.941, and PR-AUC of 0.816. It detected 50 fraud cases and produced only 4 false positives.

The MLP showed that a supervised neural network can compete on the corrected split. It detected 55 fraud cases with 12 false positives and achieved F1-score of 0.797, only slightly below Random Forest, but its PR-AUC of 0.654 was lower than the Random Forest PR-AUC.

The autoencoder achieved slightly higher recall than Random Forest, detecting 54 fraud cases, but produced 417 false positives and had much lower precision and PR-AUC. Therefore, the autoencoder is best interpreted as a high-recall anomaly detection experiment, while the Random Forest is our strongest practical model in these experiments.

7.1 Limitations and Future Work

The dataset is anonymised, so the features cannot be interpreted in detail. Also, the split used in this project is stratified and random, which is suitable for controlled model comparison but does not fully simulate real-time deployment. A future extension could evaluate time-based splits to test robustness under temporal drift.

Other possible extensions include cost-sensitive threshold selection, probability calibration, combining autoencoder reconstruction errors with supervised classifiers, and using additional contextual transaction features if available.

7.2 External Sources and Project Ownership

The dataset is an external public dataset obtained from Kaggle and originally associated with the Machine Learning Group at Universite Libre de Bruxelles. The preprocessing workflow, train/validation/test split, model training code, threshold tuning, MLP experiments, autoencoder experiments, autoencoder learning-rate sweep, evaluation plots, and final analysis were created for this project using Python libraries including pandas, scikit-learn, matplotlib, joblib, and TensorFlow/Keras. The full project repository, including corrected notebooks, source code, generated figures, and report-supporting outputs, is available at: `https://github.com/UrSnow/credit-card-fraud-detection`

Bibliography

[1] Kaggle and Machine Learning Group at Universite Libre de Bruxelles. *Credit Card Fraud Detection dataset*. `https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud`

[2] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12:2825–2830, 2011. `https://scikit-learn.org/`

[3] TensorFlow Developers. *TensorFlow and Keras documentation*. `https://www.tensorflow.org/`

[4] The pandas development team. *pandas documentation*. `https://pandas.pydata.org/`

[5] J. D. Hunter. *Matplotlib: A 2D Graphics Environment*. Computing in Science and Engineering, 9(3):90–95, 2007. `https://matplotlib.org/`